



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №3

«Інтерполяція функцій. Інтерполяційні многочлени»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Порєв В. М.*

Тема: «Інтерполяція функцій. Інтерполяційні многочлени».

Мета: Ознайомлення з інтерполяційними формулами Лагранжа, Ньютона, рекурентним співвідношенням Ейткена, методами оцінки похибки інтерполяції.

Хід роботи

Мій порядковий номер у списку групи: 6, звідси я маю інтерполювати таку функцію в таких межах за допомогою схеми Ейткена:

6	$\sin x^2 \cdot e^{-(x/2)^2}$	$[0, 3]$	1.8
---	-------------------------------	----------	-----

Рис. 1: Варіант функції та способу інтерполяції

Також додатково я маю:

1. Передбачити в програмі оцінку похибки на основі порівняння значень, отриманих за допомогою інтерполяційних многочленів різного степеню;
2. Оцінити розмитість оцінки похибки;
3. Налаштувати програму шляхом інтерполяції функції $\sin x$;
4. Результат оцінки похибки представити у вигляді графіка.

І так, опишемо словесно алгоритм методу Ейткена:

Метод Ейткена є рекурентним способом обчислення значення інтерполяційного многочлена в заданій точці x . Основна перевага цього методу полягає в тому, що він дозволяє не виписувати явно весь інтерполяційний многочлен у алгебраїчному вигляді, а безпосередньо обчислювати його значення в потрібній точці.

Метод базується на послідовному об'єднанні інтерполяційних многочленів нижчих степенів у многочлени вищих степенів. Спочатку беруться значення функції у вузлах, потім за ними будуються лінійні інтерполяції, далі квадратичні, кубічні і так далі, аж до многочлена найвищого степеня.

Початкові значення задаються як

$$P_{i,i}(x) = y_i = f(x_i), \quad i = 0, 1, \dots, n.$$

Далі для $i < j$ значення інтерполяційного многочлена обчислюється за рекурентною формулою:

$$P_{i,j}(x) = \frac{(x - x_i)P_{i+1,j}(x) - (x - x_j)P_{i,j-1}(x)}{x_j - x_i}.$$

Тут:

- $P_{i,j}(x)$ — значення інтерполяційного многочлена, побудованого за вузлами $x_i, x_{i+1}, \dots, x_j$;
- $P_{i,i}(x)$ — початкові значення, що дорівнюють значенням функції у вузлах;
- $P_{0,n}(x)$ — остаточне значення інтерполяційного многочлена, побудованого за всіма вузлами.

Таким чином, результатом застосування методу Ейткена є число

$$P_{0,n}(x),$$

яке є наближеним значенням функції $f(x)$ у вибраній точці.

Принцип методу Ейткена полягає у послідовному підвищенні степеня інтерполяційного многочлена. На нульовому кроці маємо лише значення функції у вузлах:

$$P_{i,i}(x) = y_i.$$

Потім із двох сусідніх значень будуються многочлени першого степеня:

$$P_{i,i+1}(x).$$

Далі з них будуються многочлени другого степеня:

$$P_{i,i+2}(x),$$

і так далі, поки не буде отримано

$$P_{0,n}(x).$$

Отже, метод має трикутну схему обчислень: кожен новий елемент таблиці визначається через два раніше знайдені елементи.

1. Задати відрізок інтерполювання та вибрати вузли

$$x_0, x_1, \dots, x_n.$$

2. Обчислити значення функції у вузлах:

$$y_i = f(x_i), \quad i = 0, 1, \dots, n.$$

3. Для заданої точки x покласти початкові значення

$$P_{i,i}(x) = y_i.$$

4. Послідовно обчислювати значення

$$P_{i,j}(x)$$

за рекурентною формулою Ейткена, починаючи з многочленів першого степеня і закінчуючи многочленом степеня n .

5. Остаточним результатом вважати значення

$$P_{0,n}(x).$$

6. Для аналізу точності порівняти значення, отримані для многочленів різних степенів.

Якщо використовується $n + 1$ вузлів інтерполяції, то будується інтерполяційний многочлен степеня не вище n . Наприклад:

- за 2 вузлами будується многочлен 1-го степеня;
- за 3 вузлами — многочлен 2-го степеня;
- за 11 вузлами — многочлен 10-го степеня.

Тому в таблиці обчислень величина n зазвичай означає саме степінь інтерполяційного многочлена.

Оскільки інтерполяційний многочлен лише наближує функцію між вузлами, виникає похибка інтерполяції. Її можна визначити як різницю між точним значенням функції та значенням інтерполяційного многочлена:

$$R_n(x) = f(x) - P_n(x).$$

Абсолютна похибка дорівнює

$$|R_n(x)| = |f(x) - P_n(x)|.$$

Якщо точне значення функції відоме, то це є найзручнішою формулою для обчислення реальної похибки.

Для інтерполяційного многочлена степеня n теоретична похибка має вигляд

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-x_0)(x-x_1) \cdots (x-x_n),$$

де ξ — деяка точка з проміжку, що містить вузли інтерполяції та точку x .

Ця формула показує, що величина похибки залежить від:

- похідної порядку $n+1$ від функції;
- кількості та розташування вузлів;
- положення точки x відносно вузлів.

На практиці ця формула не завжди зручна для числового обчислення, оскільки значення ξ невідоме.

У чисельних методах часто використовують практичну оцінку похибки на основі порівняння результатів, отриманих для многочленів сусідніх степенів:

$$\varepsilon_n(x) \approx |P_n(x) - P_{n-1}(x)|.$$

Така оцінка базується на тому, що при збільшенні степеня інтерполяційного многочлена наближення має стабілізуватися, а різниця між сусідніми наближеннями стає малою.

Якщо

$$|P_n(x) - P_{n-1}(x)|$$

мале, то це свідчить про те, що значення інтерполяції вже майже не змінюється і наближення є достатньо точним.

Крім абсолютної похибки, інколи розглядають відносну похибку:

$$\delta_n(x) = \frac{|f(x) - P_n(x)|}{|f(x)|}, \quad f(x) \neq 0.$$

У відсотках вона записується так:

$$\delta_n(x) \cdot 100\%.$$

Відносна похибка показує, наскільки велика помилка порівняно з самим значенням функції.

Похибка інтерполяції може збільшуватися з таких причин:

- функція має складну поведінку на відрізку;

- вузли розташовані невдало;
- точка x знаходиться поблизу краю інтервалу;
- використовується многочлен занадто високого степеня, що може призводити до осциляцій.

Тому на практиці важливо не лише побудувати інтерполяцію, а й оцінити її точність. Для реалізації алгоритмів та GUI середовища я використовуватиму мову програмування Python і інструмент Tkinter та бібліотеку customtkinter.

Програмний код для реалізації метода Ейткена для інтерполювання:

(app.py)

```

1 from src.gui import main
2
3 if __name__ == "__main__":
4
5     main()
```

(src/aitken.py)

```

1 from __future__ import annotations
2
3 def build_nodes(a: float, b: float, parts: int = 10) -> list[float]:
4     h = (b - a) / parts
5     return [a + i * h for i in range(parts + 1)]
6
7 def calculate_function_values(func, x_nodes: list[float]) -> list[float]:
8     return [func(x) for x in x_nodes]
9
10 def interpolate_full(x_nodes: list[float], y_nodes: list[float], x: float) -> float:
11     table = aitken_table(x_nodes, y_nodes, x)
12     return table[0][len(x_nodes) - 1]
13
14 def aitken_table(x_nodes: list[float], y_nodes: list[float], x: float) ->
15     ↪ list[list[float]]:
16     n = len(x_nodes)
17     p = [[0.0 for _ in range(n)] for _ in range(n)]
18
19     for i in range(n):
20         p[i][0] = y_nodes[i]
21
22     for j in range(1, n):
23         for i in range(n - j):
24             denominator = x_nodes[i + j] - x_nodes[i]
25             if abs(denominator) < 1e-15:
26                 raise ZeroDivisionError("Zero denominator in Aitken scheme.")
27             p[i][j] = (
28                 (x - x_nodes[i]) * p[i + 1][j - 1]
29                 - (x - x_nodes[i + j]) * p[i][j - 1]
30             ) / denominator
```

```

31     return p
32
33 def aitken_by_degree(x_nodes: list[float], y_nodes: list[float], x: float) ->
    ↪ list[float]:
34     approximations: list[float] = []
35
36     for degree in range(len(x_nodes)):
37         sub_x = x_nodes[: degree + 1]
38         sub_y = y_nodes[: degree + 1]
39         table = aitken_table(sub_x, sub_y, x)
40         approximations.append(table[0][degree])
41
42     return approximations

```

(src/error_analysis.py)

```

1  from __future__ import annotations
2
3  from src.aitken import aitken_by_degree
4  from src.models import ErrorRow
5
6  def compute_error_rows(func, x_nodes: list[float], y_nodes: list[float], x_value:
    ↪ float) -> tuple[float, list[ErrorRow]]:
7      true_value = func(x_value)
8      approximations = aitken_by_degree(x_nodes, y_nodes, x_value)
9
10     rows: list[ErrorRow] = []
11     for degree, approx in enumerate(approximations):
12         estimate = None if degree == 0 else abs(approximations[degree] -
    ↪ approximations[degree - 1])
13         actual_error = abs(true_value - approx)
14         rows.append(
15             ErrorRow(
16                 degree=degree,
17                 approximation=approx,
18                 estimate=estimate,
19                 actual_error=actual_error,
20             )
21         )
22     return true_value, rows

```

(src/functions.py)

```

1  from __future__ import annotations
2
3  import math
4  from typing import Callable
5
6  def target_function(x: float) -> float:
7      return math.sin(x ** 2) * math.exp(-((x / 2) ** 2))
8
9  def test_function(x: float) -> float:
10     return math.sin(x)

```

```

11
12 def get_function_by_name(name: str) -> tuple[Callable[[float], float], str]:
13     if name == "test":
14         return test_function, "sin(x)"
15     return target_function, "sin(x^2) * exp(-(x/2)^2)"

```

(src/graph_build.py)

```

1 from __future__ import annotations
2
3 from matplotlib.figure import Figure
4
5 from src.aitken import interpolate_full
6
7
8 def build_interpolation_figure(
9     func,
10     x_nodes: list[float],
11     y_nodes: list[float],
12     a: float,
13     b: float,
14     x_value: float,
15     plot_points: int,
16 ) -> Figure:
17     x_dense = [a + (b - a) * i / (plot_points - 1) for i in range(plot_points)]
18     y_true = [func(x) for x in x_dense]
19     y_interp = [interpolate_full(x_nodes, y_nodes, x) for x in x_dense]
20     y_error = [abs(y1 - y2) for y1, y2 in zip(y_true, y_interp)]
21
22     true_at_x = func(x_value)
23     interp_at_x = interpolate_full(x_nodes, y_nodes, x_value)
24
25     fig = Figure(figsize=(8.2, 5.8), dpi=100)
26     ax = fig.add_subplot(111)
27
28     ax.plot(x_dense, y_true, label="f(x)")
29     ax.plot(x_dense, y_interp, label="Aitken interpolation")
30     ax.plot(x_dense, y_error, label="|f(x)-P(x)|")
31     ax.scatter(x_nodes, y_nodes, label="Nodes")
32     ax.scatter([x_value], [true_at_x], label="f(x*)")
33     ax.scatter([x_value], [interp_at_x], label="P(x*)")
34
35     ax.set_title("Aitken interpolation graph")
36     ax.set_xlabel("x")
37     ax.set_ylabel("y")
38     ax.grid(True)
39     ax.legend()
40     fig.tight_layout()
41
42     return fig

```

(src/gui.py)

```

1  from __future__ import annotations
2
3  import tkinter as tk
4  from dataclasses import dataclass
5  from tkinter import filedialog, ttk
6
7  import customtkinter as ctk
8  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
9
10 from src.aitken import build_nodes, calculate_function_values
11 from src.error_analysis import compute_error_rows
12 from src.functions import get_function_by_name
13 from src.graph_build import build_interpolation_figure
14 from src.io_utils import load_json_config
15 from src.models import PageConfig
16 from src.report import build_full_report, build_short_result
17 from src.theme import MAIN_THEME
18
19
20 @dataclass
21 class InterpolationState:
22     function_name: str = "target"
23     function_title: str = "sin(x^2) * exp(-(x/2)^2)"
24     a: float = 0.0
25     b: float = 3.0
26     x_value: float = 1.5
27     parts: int = 10
28     plot_points: int = 400
29     x_nodes: list[float] | None = None
30     y_nodes: list[float] | None = None
31     true_value: float | None = None
32     rows: list | None = None
33     report: str = ""
34
35
36 class BasePage(ctk.CTkFrame):
37     def __init__(
38         self,
39         master: ctk.CTkFrame,
40         page_config: PageConfig,
41         fonts: dict[str, ctk.CTkFont],
42     ) -> None:
43         super().__init__(master, corner_radius=0)
44         self.page_config = page_config
45         self.fonts = fonts
46         self.grid_columnconfigure(0, weight=1)
47
48     def apply_theme(self, palette: dict[str, str]) -> None:
49         self.configure(fg_color=palette["bg"])
50
51
52 class MainPage(BasePage):
53     def __init__(

```



```

54         self,
55         master: ctk.CTkFrame,
56         page_config: PageConfig,
57         fonts: dict[str, ctk.CTkFont],
58         app_state: InterpolationState,
59     ) -> None:
60         super().__init__(master, page_config, fonts)
61         self.app_state = app_state
62
63         self.result_var = tk.StringVar(value="Ready")
64         self.error_var = tk.StringVar(value="")
65
66         self.grid_rowconfigure(3, weight=1)
67         self.grid_columnconfigure(0, weight=1)
68
69         self._build_header()
70         self._build_form()
71         self._build_actions()
72         self._build_output()
73         self.reset_defaults()
74
75     def _build_header(self) -> None:
76         self.header_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
77         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
78         self.header_card.grid_columnconfigure(0, weight=1)
79
80         self.title_label = ctk.CTkLabel(
81             self.header_card,
82             text=self.page_config.title,
83             anchor="w",
84             font=self.fonts["title"],
85         )
86         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
87
88         self.subtitle_label = ctk.CTkLabel(
89             self.header_card,
90             text=self.page_config.subtitle,
91             anchor="w",
92             font=self.fonts["subtitle"],
93         )
94         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
95             ↪ 12))
96
97     def _build_form(self) -> None:
98         self.form_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
99         self.form_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
100         self.form_card.grid_columnconfigure((0, 1), weight=1)
101
102         self.function_label = ctk.CTkLabel(
103             self.form_card,
104             text="Function:",
105             anchor="w",
106             font=self.fonts["label"],
107         )

```

```

107     self.function_label.grid(row=0, column=0, sticky="w", padx=20, pady=(14, 6))
108
109     self.function_menu = ctk.CTkOptionMenu(
110         self.form_card,
111         values=["target", "test"],
112         font=self.fonts["body"],
113         command=self._on_function_change,
114     )
115     self.function_menu.grid(row=1, column=0, sticky="ew", padx=(20, 8), pady=(0,
116 ↪ 12))
117
118     self.load_button = ctk.CTkButton(
119         self.form_card,
120         text="Load JSON",
121         height=40,
122         corner_radius=11,
123         font=self.fonts["button"],
124         command=self.load_json,
125     )
126     self.load_button.grid(row=1, column=1, sticky="ew", padx=(8, 20), pady=(0,
127 ↪ 12))
128
129     self.entry_a = self._build_labeled_entry(self.form_card, 2, 0, "a:")
130     self.entry_b = self._build_labeled_entry(self.form_card, 2, 1, "b:")
131     self.entry_x = self._build_labeled_entry(self.form_card, 4, 0, "x:")
132     self.entry_parts = self._build_labeled_entry(self.form_card, 4, 1, "Parts:")
133     self.entry_plot_points = self._build_labeled_entry(self.form_card, 6, 0,
134 ↪ "Plot points:")
135
136     def _build_labeled_entry(
137         self,
138         parent: ctk.CTkFrame,
139         row: int,
140         column: int,
141         label_text: str,
142     ) -> ctk.CTkEntry:
143         label = ctk.CTkLabel(
144             parent,
145             text=label_text,
146             anchor="w",
147             font=self.fonts["label"],
148         )
149         label.grid(row=row, column=column, sticky="w", padx=20, pady=(0, 6))
150
151         entry = ctk.CTkEntry(
152             parent,
153             height=40,
154             corner_radius=10,
155             font=self.fonts["body"],
156         )
157         entry.grid(row=row + 1, column=column, sticky="ew", padx=20, pady=(0, 12))
158         return entry
159
160     def _build_actions(self) -> None:

```

```

158 self.actions = ctk.CTkFrame(self, fg_color="transparent")
159 self.actions.grid(row=2, column=0, sticky="ew", pady=(0, 10))
160 self.actions.grid_columnconfigure((0, 1, 2), weight=1)
161
162 self.compute_button = ctk.CTkButton(
163     self.actions,
164     text="Compute",
165     height=40,
166     corner_radius=11,
167     font=self.fonts["button"],
168     command=self.compute,
169 )
170 self.compute_button.grid(row=0, column=0, sticky="ew", padx=(0, 7))
171
172 self.save_button = ctk.CTkButton(
173     self.actions,
174     text="Save Report",
175     height=40,
176     corner_radius=11,
177     font=self.fonts["button"],
178     command=self.save_report,
179 )
180 self.save_button.grid(row=0, column=1, sticky="ew", padx=7)
181
182 self.clear_button = ctk.CTkButton(
183     self.actions,
184     text="Clear",
185     height=40,
186     corner_radius=11,
187     font=self.fonts["button"],
188     command=self.clear_all,
189 )
190 self.clear_button.grid(row=0, column=2, sticky="ew", padx=(7, 0))
191
192 def _build_output(self) -> None:
193     self.output_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
194     self.output_card.grid(row=3, column=0, sticky="nsew")
195     self.output_card.grid_columnconfigure(0, weight=1)
196     self.output_card.grid_rowconfigure(4, weight=1)
197
198     self.result_title = ctk.CTkLabel(
199         self.output_card,
200         text="Result",
201         anchor="w",
202         font=self.fonts["label"],
203     )
204     self.result_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
205
206     self.result_label = ctk.CTkLabel(
207         self.output_card,
208         textvariable=self.result_var,
209         anchor="w",
210         justify="left",
211         wraplength=860,

```

```

212         font=self.fonts["result"],
213     )
214     self.result_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
215
216     self.error_label = ctk.CTkLabel(
217         self.output_card,
218         textvariable=self.error_var,
219         anchor="w",
220         justify="left",
221         wraplength=860,
222         font=self.fonts["body"],
223     )
224     self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 10))
225
226     self.table_title = ctk.CTkLabel(
227         self.output_card,
228         text="Table",
229         anchor="w",
230         font=self.fonts["label"],
231     )
232     self.table_title.grid(row=3, column=0, sticky="ew", padx=20, pady=(0, 6))
233
234     self.table_frame = tk.Frame(self.output_card, bg="#090909")
235     self.table_frame.grid(row=4, column=0, sticky="nsew", padx=20, pady=(0, 14))
236     self.table_frame.grid_rowconfigure(0, weight=1)
237     self.table_frame.grid_columnconfigure(0, weight=1)
238
239     columns = ("degree", "approximation", "estimate", "actual_error")
240     self.results_table = ttk.Treeview(
241         self.table_frame,
242         columns=columns,
243         show="headings",
244         height=11,
245     )
246
247     self.results_table.heading("degree", text="n")
248     self.results_table.heading("approximation", text="Pn(x)")
249     self.results_table.heading("estimate", text="|Pn - P(n-1)|")
250     self.results_table.heading("actual_error", text="|f(x) - Pn(x)|")
251
252     self.results_table.column("degree", width=70, anchor="center")
253     self.results_table.column("approximation", width=190, anchor="center")
254     self.results_table.column("estimate", width=190, anchor="center")
255     self.results_table.column("actual_error", width=190, anchor="center")
256
257     self.table_scrollbar = ttk.Scrollbar(
258         self.table_frame,
259         orient="vertical",
260         command=self.results_table.yview,
261     )
262     self.results_table.configure(yscrollcommand=self.table_scrollbar.set)
263
264     self.results_table.grid(row=0, column=0, sticky="nsew")
265     self.table_scrollbar.grid(row=0, column=1, sticky="ns")

```

```

266
267 def _fill_table(self, rows) -> None:
268     for item in self.results_table.get_children():
269         self.results_table.delete(item)
270
271     for row in rows:
272         estimate_text = "-" if row.estimate is None else f"{row.estimate:.10f}"
273         self.results_table.insert(
274             "",
275             "end",
276             values=(
277                 row.degree,
278                 f"{row.approximation:.10f}",
279                 estimate_text,
280                 f"{row.actual_error:.10f}",
281             ),
282         )
283
284 def _on_function_change(self, choice: str) -> None:
285     if choice == "test":
286         self.entry_a.delete(0, tk.END)
287         self.entry_a.insert(0, "0")
288         self.entry_b.delete(0, tk.END)
289         self.entry_b.insert(0, "3.1415926535")
290         self.entry_x.delete(0, tk.END)
291         self.entry_x.insert(0, "1.0")
292         self.entry_parts.delete(0, tk.END)
293         self.entry_parts.insert(0, "10")
294         self.entry_plot_points.delete(0, tk.END)
295         self.entry_plot_points.insert(0, "400")
296     else:
297         self.entry_a.delete(0, tk.END)
298         self.entry_a.insert(0, "0")
299         self.entry_b.delete(0, tk.END)
300         self.entry_b.insert(0, "3")
301         self.entry_x.delete(0, tk.END)
302         self.entry_x.insert(0, "1.5")
303         self.entry_parts.delete(0, tk.END)
304         self.entry_parts.insert(0, "10")
305         self.entry_plot_points.delete(0, tk.END)
306         self.entry_plot_points.insert(0, "400")
307
308 def load_json(self) -> None:
309     path = filedialog.askopenfilename(
310         title="Select JSON file",
311         filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
312     )
313     if not path:
314         return
315
316     try:
317         loaded = load_json_config(path)
318
319         function_name = str(loaded.get("function", "target"))

```

```

320         self.function_menu.set(function_name)
321
322         self.entry_a.delete(0, tk.END)
323         self.entry_a.insert(0, str(loader.get("a", 0)))
324
325         self.entry_b.delete(0, tk.END)
326         self.entry_b.insert(0, str(loader.get("b", 3)))
327
328         self.entry_x.delete(0, tk.END)
329         self.entry_x.insert(0, str(loader.get("x_value", 1.5)))
330
331         self.entry_parts.delete(0, tk.END)
332         self.entry_parts.insert(0, str(loader.get("parts", 10)))
333
334         self.entry_plot_points.delete(0, tk.END)
335         self.entry_plot_points.insert(0, str(loader.get("plot_points", 400)))
336
337         self.result_var.set("JSON loaded. Press Compute.")
338         self.error_var.set("")
339     except Exception as exc:
340         self.result_var.set("No result")
341         self.error_var.set(f"Error: {exc}")
342
343     def _parse_input(self) -> tuple[str, float, float, float, int, int]:
344         function_name = self.function_menu.get()
345         a = float(self.entry_a.get().strip())
346         b = float(self.entry_b.get().strip())
347         x_value = float(self.entry_x.get().strip())
348         parts = int(self.entry_parts.get().strip())
349         plot_points = int(self.entry_plot_points.get().strip())
350
351         if b <= a:
352             raise ValueError("Потрібно, щоб b > a.")
353         if parts < 1:
354             raise ValueError("Кількість частин має бути не меншою за 1.")
355         if plot_points < 50:
356             raise ValueError("Кількість точок для графіка має бути не меншою за
357                 ↪ 50.")
358         if not (a <= x_value <= b):
359             raise ValueError("Точка x повинна належати відрітку [a, b].")
360
361         return function_name, a, b, x_value, parts, plot_points
362
363     def compute(self) -> None:
364         try:
365             function_name, a, b, x_value, parts, plot_points = self._parse_input()
366             func, function_title = get_function_by_name(function_name)
367
368             x_nodes = build_nodes(a, b, parts)
369             y_nodes = calculate_function_values(func, x_nodes)
370             true_value, rows = compute_error_rows(func, x_nodes, y_nodes, x_value)
371
372             report = build_full_report(
373                 func_name=function_title,

```

```

373         a=a,
374         b=b,
375         x_value=x_value,
376         x_nodes=x_nodes,
377         y_nodes=y_nodes,
378         rows=rows,
379         true_value=true_value,
380     )
381
382     self.app_state.function_name = function_name
383     self.app_state.function_title = function_title
384     self.app_state.a = a
385     self.app_state.b = b
386     self.app_state.x_value = x_value
387     self.app_state.parts = parts
388     self.app_state.plot_points = plot_points
389     self.app_state.x_nodes = x_nodes
390     self.app_state.y_nodes = y_nodes
391     self.app_state.true_value = true_value
392     self.app_state.rows = rows
393     self.app_state.report = report
394
395     self.result_var.set(build_short_result(rows, true_value))
396     self.error_var.set("")
397     self._fill_table(rows)
398 except Exception as exc:
399     self.result_var.set("No result")
400     self.error_var.set(f"Error: {exc}")
401
402 def save_report(self) -> None:
403     if not self.app_state.report.strip():
404         self.result_var.set("No result")
405         self.error_var.set("Error: Спочатку виконай обчислення.")
406         return
407
408     path = filedialog.asksaveasfilename(
409         title="Save report",
410         defaultextension=".txt",
411         filetypes=[("Text files", "*.txt"), ("All files", "*.*")],
412     )
413     if not path:
414         return
415
416     try:
417         with open(path, "w", encoding="utf-8") as file:
418             file.write(self.app_state.report)
419         self.error_var.set("")
420     except Exception as exc:
421         self.error_var.set(f"Error: {exc}")
422
423 def clear_all(self) -> None:
424     self.reset_defaults()
425
426 def reset_defaults(self) -> None:

```

```

427         self.function_menu.set("target")
428
429         self.entry_a.delete(0, tk.END)
430         self.entry_a.insert(0, "0")
431
432         self.entry_b.delete(0, tk.END)
433         self.entry_b.insert(0, "3")
434
435         self.entry_x.delete(0, tk.END)
436         self.entry_x.insert(0, "1.5")
437
438         self.entry_parts.delete(0, tk.END)
439         self.entry_parts.insert(0, "10")
440
441         self.entry_plot_points.delete(0, tk.END)
442         self.entry_plot_points.insert(0, "400")
443
444         self.result_var.set("Ready")
445         self.error_var.set("")
446
447         for item in self.results_table.get_children():
448             self.results_table.delete(item)
449
450         self.app_state.function_name = "target"
451         self.app_state.function_title = "sin(x^2) * exp(-(x/2)^2)"
452         self.app_state.a = 0.0
453         self.app_state.b = 3.0
454         self.app_state.x_value = 1.5
455         self.app_state.parts = 10
456         self.app_state.plot_points = 400
457         self.app_state.x_nodes = None
458         self.app_state.y_nodes = None
459         self.app_state.true_value = None
460         self.app_state.rows = None
461         self.app_state.report = ""
462
463     def apply_theme(self, palette: dict[str, str]) -> None:
464         super().apply_theme(palette)
465
466         for card in (self.header_card, self.form_card, self.output_card):
467             card.configure(
468                 fg_color=palette["panel"],
469                 border_color=palette["surface_soft"],
470             )
471
472         self.title_label.configure(text_color=palette["text"])
473         self.subtitle_label.configure(text_color=palette["muted"])
474         self.function_label.configure(text_color=palette["muted"])
475
476         for entry in (
477             self.entry_a,
478             self.entry_b,
479             self.entry_x,
480             self.entry_parts,

```



```

481         self.entry_plot_points,
482     ):
483         entry.configure(
484             fg_color=palette["surface"],
485             border_color=palette["surface_soft"],
486             text_color=palette["text"],
487         )
488
489     self.function_menu.configure(
490         fg_color=palette["segment_active"],
491         button_color=palette["segment"],
492         button_hover_color=palette["segment_hover"],
493         text_color=palette["text"],
494     )
495
496     self.compute_button.configure(
497         fg_color=palette["accent"],
498         hover_color=palette["accent_hover"],
499         text_color=palette["accent_text"],
500     )
501
502     self.load_button.configure(
503         fg_color=palette["segment"],
504         hover_color=palette["segment_hover"],
505         text_color=palette["text"],
506     )
507
508     self.save_button.configure(
509         fg_color=palette["segment"],
510         hover_color=palette["segment_hover"],
511         text_color=palette["text"],
512     )
513
514     self.clear_button.configure(
515         fg_color=palette["segment"],
516         hover_color=palette["segment_hover"],
517         text_color=palette["text"],
518     )
519
520     self.result_title.configure(text_color=palette["muted"])
521     self.table_title.configure(text_color=palette["muted"])
522     self.result_label.configure(text_color=palette["text"])
523     self.error_label.configure(text_color=palette["error"])
524
525     style = ttk.Style()
526     style.theme_use("default")
527
528     style.configure(
529         "Treeview",
530         background="#111111",
531         foreground="#F5F7FA",
532         fieldbackground="#111111",
533         rowheight=28,
534         bordercolor="#1B1B1B",
535         borderwidth=1,
536     )
537     style.configure(

```

```

535         "Treeview.Heading",
536         background="#131313",
537         foreground="#F5F7FA",
538         relief="flat",
539     )
540     style.map(
541         "Treeview",
542         background=[("selected", "#1F2D3C")],
543         foreground=[("selected", "#F5F7FA")],
544     )
545
546
547 class GraphPage(BasePage):
548     def __init__(
549         self,
550         master: ctk.CTkFrame,
551         page_config: PageConfig,
552         fonts: dict[str, ctk.CTkFont],
553         app_state: InterpolationState,
554     ) -> None:
555         super().__init__(master, page_config, fonts)
556         self.app_state = app_state
557
558         self.status_var = tk.StringVar(value="Ready to build graph")
559         self.error_var = tk.StringVar(value="")
560         self.canvas: FigureCanvasTkAgg | None = None
561
562         self.grid_rowconfigure(3, weight=1)
563
564         self._build_header()
565         self._build_controls()
566         self._build_output()
567         self._build_plot_area()
568
569     def _build_header(self) -> None:
570         self.header_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
571         self.header_card.grid(row=0, column=0, sticky="ew", pady=(0, 14))
572         self.header_card.grid_columnconfigure(0, weight=1)
573
574         self.title_label = ctk.CTkLabel(
575             self.header_card,
576             text=self.page_config.title,
577             anchor="w",
578             font=self.fonts["title"],
579         )
580         self.title_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 4))
581
582         self.subtitle_label = ctk.CTkLabel(
583             self.header_card,
584             text=self.page_config.subtitle,
585             anchor="w",
586             font=self.fonts["subtitle"],
587         )
588         self.subtitle_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0,
↵ 12))

```

```

589
590 def _build_controls(self) -> None:
591     self.controls_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
592     self.controls_card.grid(row=1, column=0, sticky="ew", pady=(0, 14))
593     self.controls_card.grid_columnconfigure(0, weight=1)
594
595     self.info_label = ctk.CTkLabel(
596         self.controls_card,
597         text=(
598             "Побудова графіка функції, повної інтерполяції за схемою Ейткена "
599             "та абсолютної похибки  $|f(x)-P(x)|$ ."
600         ),
601         anchor="w",
602         justify="left",
603         wraplength=860,
604         font=self.fonts["body"],
605     )
606     self.info_label.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 12))
607
608     self.build_button = ctk.CTkButton(
609         self.controls_card,
610         text="Build Graph",
611         height=40,
612         corner_radius=11,
613         font=self.fonts["button"],
614         command=self.build_graph,
615     )
616     self.build_button.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 14))
617
618 def _build_output(self) -> None:
619     self.output_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
620     self.output_card.grid(row=2, column=0, sticky="ew")
621     self.output_card.grid_columnconfigure(0, weight=1)
622
623     self.status_title = ctk.CTkLabel(
624         self.output_card,
625         text="Status",
626         anchor="w",
627         font=self.fonts["label"],
628     )
629     self.status_title.grid(row=0, column=0, sticky="ew", padx=20, pady=(14, 6))
630
631     self.status_label = ctk.CTkLabel(
632         self.output_card,
633         textvariable=self.status_var,
634         anchor="w",
635         justify="left",
636         wraplength=860,
637         font=self.fonts["result"],
638     )
639     self.status_label.grid(row=1, column=0, sticky="ew", padx=20, pady=(0, 8))
640
641     self.error_label = ctk.CTkLabel(
642         self.output_card,

```

```

643         textvariable=self.error_var,
644         anchor="w",
645         justify="left",
646         wraplength=860,
647         font=self.fonts["body"],
648     )
649     self.error_label.grid(row=2, column=0, sticky="ew", padx=20, pady=(0, 14))
650
651     def _build_plot_area(self) -> None:
652         self.plot_card = ctk.CTkFrame(self, corner_radius=14, border_width=1)
653         self.plot_card.grid(row=3, column=0, sticky="nsew", pady=(14, 0))
654         self.plot_card.grid_columnconfigure(0, weight=1)
655         self.plot_card.grid_rowconfigure(0, weight=1)
656
657         self.plot_container = ctk.CTkFrame(self.plot_card, fg_color="transparent")
658         self.plot_container.grid(row=0, column=0, sticky="nsew", padx=12, pady=12)
659         self.plot_container.grid_columnconfigure(0, weight=1)
660         self.plot_container.grid_rowconfigure(0, weight=1)
661
662     def build_graph(self) -> None:
663         try:
664             if self.app_state.x_nodes is None or self.app_state.y_nodes is None:
665                 raise ValueError("Спочатку виконай обчислення на сторінці Main.")
666
667             self.status_var.set("Building graph...")
668             self.error_var.set("")
669             self.update_idletasks()
670
671             func, _ = get_function_by_name(self.app_state.function_name)
672             fig = build_interpolation_figure(
673                 func=func,
674                 x_nodes=self.app_state.x_nodes,
675                 y_nodes=self.app_state.y_nodes,
676                 a=self.app_state.a,
677                 b=self.app_state.b,
678                 x_value=self.app_state.x_value,
679                 plot_points=self.app_state.plot_points,
680             )
681
682             if self.canvas is not None:
683                 self.canvas.get_tk_widget().destroy()
684
685             self.canvas = FigureCanvasTkAgg(fig, master=self.plot_container)
686             self.canvas.draw()
687             self.canvas.get_tk_widget().grid(row=0, column=0, sticky="nsew")
688
689             self.status_var.set("Graph was built successfully")
690         except Exception as exc:
691             self.status_var.set("Graph was not built")
692             self.error_var.set(f"Error: {exc}")
693
694     def apply_theme(self, palette: dict[str, str]) -> None:
695         super().apply_theme(palette)
696

```

```

697     for card in (self.header_card, self.controls_card, self.output_card,
698 ↪ self.plot_card):
699         card.configure(
700             fg_color=palette["panel"],
701             border_color=palette["surface_soft"],
702         )
703
704     self.title_label.configure(text_color=palette["text"])
705     self.subtitle_label.configure(text_color=palette["muted"])
706     self.info_label.configure(text_color=palette["muted"])
707
708     self.build_button.configure(
709         fg_color=palette["accent"],
710         hover_color=palette["accent_hover"],
711         text_color=palette["accent_text"],
712     )
713
714     self.status_title.configure(text_color=palette["muted"])
715     self.status_label.configure(text_color=palette["text"])
716     self.error_label.configure(text_color=palette["error"])
717
718 class AlgorithmsApp(ctlk.CTk):
719     def __init__(self) -> None:
720         super().__init__()
721         ctlk.set_appearance_mode("dark")
722
723         self.title("AitkenInterpolationApp")
724         self.resizable(False, False)
725         self.geometry("720x900")
726
727         self.current_page = "main"
728         self.nav_buttons: dict[str, ctlk.CTkButton] = {}
729         self.pages: dict[str, BasePage] = {}
730         self.app_state = InterpolationState()
731
732         self.fonts: dict[str, ctlk.CTkFont] = {
733             "title": ctlk.CTkFont(family="Segoe UI", size=27, weight="bold"),
734             "subtitle": ctlk.CTkFont(family="Segoe UI", size=13),
735             "label": ctlk.CTkFont(family="Segoe UI", size=13, weight="bold"),
736             "body": ctlk.CTkFont(family="Segoe UI", size=12),
737             "button": ctlk.CTkFont(family="Segoe UI", size=12, weight="bold"),
738             "result": ctlk.CTkFont(family="Consolas", size=16, weight="bold"),
739             "nav": ctlk.CTkFont(family="Segoe UI", size=12, weight="bold"),
740         }
741
742         self.page_configs: tuple[PageConfig, ...] = (
743             PageConfig(
744                 page_id="main",
745                 nav_title="Main",
746                 title="Aitken Interpolation",
747                 subtitle="Обчислення інтерполяції функції та оцінки похибки",
748             ),
749             PageConfig(

```

```

750         page_id="graph",
751         nav_title="Graph",
752         title="Aitken Interpolation Graph",
753         subtitle="Графік функції, інтерполяції та абсолютної похибки",
754     ),
755 )
756
757 self.grid_columnconfigure(0, weight=1)
758 self.grid_rowconfigure(1, weight=1)
759
760 self._build_topbar()
761 self._build_pages()
762 self._apply_theme()
763 self.show_page(self.current_page)
764
765 def _build_topbar(self) -> None:
766     self.topbar = ctk.CTkFrame(self, corner_radius=0)
767     self.topbar.grid(row=0, column=0, sticky="ew")
768     self.topbar.grid_columnconfigure(0, weight=1)
769
770     self.nav_frame = ctk.CTkFrame(self.topbar, fg_color="transparent")
771     self.nav_frame.grid(row=0, column=0, sticky="w", padx=20, pady=(18, 12))
772
773     for index, page in enumerate(self.page_configs):
774         button = ctk.CTkButton(
775             self.nav_frame,
776             text=page.nav_title,
777             width=110,
778             height=36,
779             corner_radius=10,
780             font=self.fonts["nav"],
781             border_width=1,
782             command=lambda pid=page.page_id: self.show_page(pid),
783         )
784         button.grid(row=0, column=index, padx=(0, 8))
785         self.nav_buttons[page.page_id] = button
786
787 def _build_pages(self) -> None:
788     self.page_host = ctk.CTkFrame(self, corner_radius=0)
789     self.page_host.grid(row=1, column=0, sticky="nsew", padx=20, pady=(0, 20))
790     self.page_host.grid_rowconfigure(0, weight=1)
791     self.page_host.grid_columnconfigure(0, weight=1)
792
793     main_page = MainPage(
794         self.page_host,
795         self.page_configs[0],
796         self.fonts,
797         self.app_state,
798     )
799     main_page.grid(row=0, column=0, sticky="nsew")
800     self.pages["main"] = main_page
801
802     graph_page = GraphPage(
803         self.page_host,

```

```

804         self.page_configs[1],
805         self.fonts,
806         self.app_state,
807     )
808     graph_page.grid(row=0, column=0, sticky="nsew")
809     self.pages["graph"] = graph_page
810
811     def show_page(self, page_id: str) -> None:
812         self.current_page = page_id
813         self.pages[page_id].tkraise()
814         self._refresh_nav_buttons()
815
816     def _refresh_nav_buttons(self) -> None:
817         palette = MAIN_THEME
818         for page_id, button in self.nav_buttons.items():
819             if page_id == self.current_page:
820                 button.configure(
821                     fg_color=palette["segment_active"],
822                     hover_color=palette["segment_active"],
823                     text_color=palette["text"],
824                     border_color=palette["segment_active"],
825                 )
826             else:
827                 button.configure(
828                     fg_color=palette["segment"],
829                     hover_color=palette["segment_hover"],
830                     text_color=palette["muted"],
831                     border_color=palette["surface_soft"],
832                 )
833
834     def _apply_theme(self) -> None:
835         palette = MAIN_THEME
836
837         self.configure(fg_color=palette["bg"])
838         self.topbar.configure(fg_color=palette["topbar"])
839         self.page_host.configure(fg_color=palette["bg"])
840
841         for page in self.pages.values():
842             page.apply_theme(palette)
843
844         self._refresh_nav_buttons()
845
846
847     def main() -> None:
848         app = AlgorithmsApp()
849         app.mainloop()

```

(src/io_utils.py)

```

1  from __future__ import annotations
2
3  from matplotlib.figure import Figure
4
5  from src.aitken import interpolate_full

```

```

6
7
8 def build_interpolation_figure(
9     func,
10     x_nodes: list[float],
11     y_nodes: list[float],
12     a: float,
13     b: float,
14     x_value: float,
15     plot_points: int,
16 ) -> Figure:
17     x_dense = [a + (b - a) * i / (plot_points - 1) for i in range(plot_points)]
18     y_true = [func(x) for x in x_dense]
19     y_interp = [interpolate_full(x_nodes, y_nodes, x) for x in x_dense]
20     y_error = [abs(y1 - y2) for y1, y2 in zip(y_true, y_interp)]
21
22     true_at_x = func(x_value)
23     interp_at_x = interpolate_full(x_nodes, y_nodes, x_value)
24
25     fig = Figure(figsize=(8.2, 5.8), dpi=100)
26     ax = fig.add_subplot(111)
27
28     ax.plot(x_dense, y_true, label="f(x)")
29     ax.plot(x_dense, y_interp, label="Aitken interpolation")
30     ax.plot(x_dense, y_error, label="|f(x)-P(x)|")
31     ax.scatter(x_nodes, y_nodes, label="Nodes")
32     ax.scatter([x_value], [true_at_x], label="f(x*)")
33     ax.scatter([x_value], [interp_at_x], label="P(x*)")
34
35     ax.set_title("Aitken interpolation graph")
36     ax.set_xlabel("x")
37     ax.set_ylabel("y")
38     ax.grid(True)
39     ax.legend()
40     fig.tight_layout()
41
42     return fig

```

(src/models.py)

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5
6 @dataclass(frozen=True)
7 class PageConfig:
8     page_id: str
9     nav_title: str
10    title: str
11    subtitle: str
12
13
14 @dataclass(frozen=True)

```



```

15 class ErrorRow:
16     degree: int
17     approximation: float
18     estimate: float | None
19     actual_error: float

```

(src/report.py)

```

1  from __future__ import annotations
2
3  from src.models import ErrorRow
4
5
6  def format_float(value: float | None, digits: int = 10) -> str:
7      if value is None:
8          return "-"
9      return f"{value:.{digits}f}"
10
11
12  def build_short_result(rows: list[ErrorRow], true_value: float) -> str:
13      best = rows[-1]
14      return (
15          f"f(x) = {true_value:.10f}\n"
16          f"P_{best.degree}(x) = {best.approximation:.10f}\n"
17          f"Estimate = {format_float(best.estimate, 10)}\n"
18          f"Actual error = {best.actual_error:.10f}"
19      )
20
21
22  def build_full_report(
23      func_name: str,
24      a: float,
25      b: float,
26      x_value: float,
27      x_nodes: list[float],
28      y_nodes: list[float],
29      rows: list[ErrorRow],
30      true_value: float,
31  ) -> str:
32      lines: list[str] = []
33
34      lines.append("Aitken interpolation report")
35      lines.append("=" * 78)
36      lines.append(f"Function: {func_name}")
37      lines.append(f"Interval: [{a}, {b}]")
38      lines.append(f"x = {x_value}")
39      lines.append(f"f(x) = {true_value:.12f}")
40      lines.append("")
41
42      lines.append("Nodes:")
43      lines.append("i          x_i          y_i")
44      lines.append("-" * 48)
45      for i, (x_i, y_i) in enumerate(zip(x_nodes, y_nodes)):
46          lines.append(f"{i:<8}{x_i:<18.10f}{y_i:<18.10f}")

```

```

47
48     lines.append("")
49     lines.append("Errors by polynomial degree:")
50     lines.append("n          P_n(x)          |P_n - P_(n-1)|          |f(x)-P_n(x)|")
51     lines.append("-" * 78)
52     for row in rows:
53         lines.append(
54             f"{row.degree:<8}"
55             f"{row.approximation:<20.10f}"
56             f"{format_float(row.estimate, 10):<20}"
57             f"{row.actual_error:<20.10f}"
58         )
59
60     return "\n".join(lines)

```

(src/theme.py)

```

1  MAIN_THEME: dict[str, str] = {
2      "bg": "#000000",
3      "topbar": "#000000",
4      "panel": "#090909",
5      "surface": "#111111",
6      "surface_soft": "#1B1B1B",
7      "text": "#F5F7FA",
8      "muted": "#97A1AD",
9      "accent": "#29A8FF",
10     "accent_hover": "#44B4FF",
11     "accent_text": "#06121B",
12     "segment": "#131313",
13     "segment_hover": "#1E1E1E",
14     "segment_active": "#1F2D3C",
15     "error": "#FF5B71",
16 }

```

Скріншоти демонстрації програми:

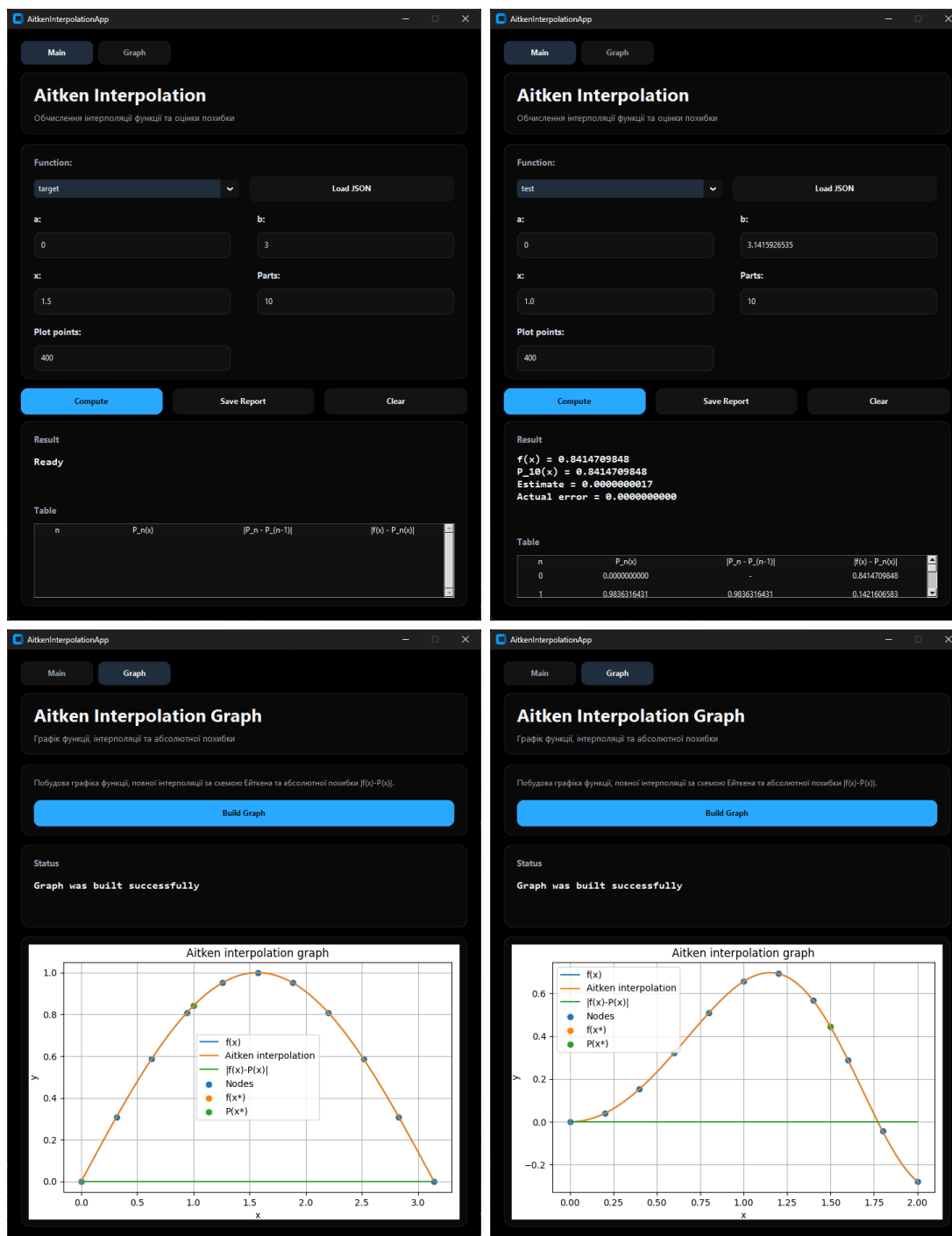


Рис. 2: Скріншоти демонстрації програми

Аналіз результатів

У ході виконання лабораторної роботи було реалізовано програму для інтерполяції функції за схемою Ейткена, а також засоби чисельної оцінки похибки та її графічного аналізу. Отримані результати дозволяють зробити не лише формальний висновок про працездатність програми, а й провести змістовний аналіз поведінки інтерполяційного процесу.

Перш за все, результати обчислень показують, що схема Ейткена коректно відтворює значення інтерполяційного многочлена в заданій точці без необхідності явно будувати

його алгебраїчний запис. Це є важливою практичною перевагою методу, оскільки при використанні великої кількості вузлів явний вигляд многочлена стає громіздким, тоді як рекурентна схема дозволяє отримати потрібне числове значення безпосередньо.

У програмі для кожного степеня інтерполяційного многочлена було обчислено значення $P_n(x)$, починаючи від нульового степеня і закінчуючи многочленом, побудованим за всіма вузлами. Такий підхід дав змогу простежити, як саме змінюється наближення при зростанні степеня многочлена. Якщо значення $P_n(x)$ при збільшенні n стабілізується, це означає, що інтерполяційний процес є стійким у вибраній точці, а додавання нових вузлів уже не дає суттєвої зміни результату. Саме така поведінка і є ознакою того, що інтерполяція в цій точці є достатньо точною.

Окрему увагу було приділено оцінці похибки. У роботі використовувались дві величини:

$$|f(x) - P_n(x)|$$

та

$$|P_n(x) - P_{n-1}(x)|.$$

Перша величина є реальною абсолютною похибкою, оскільки показує безпосереднє відхилення інтерполяційного значення від точного значення функції. Друга величина є практичною оцінкою похибки, побудованою на різниці між двома сусідніми наближеннями. Якщо ця різниця мала, то можна зробити висновок, що процес інтерполяції збігається і подальше підвищення степеня многочлена вже не дає істотного покращення.

Аналіз таблиці значень показує, що оцінка

$$|P_n(x) - P_{n-1}(x)|$$

не є точною похибкою у строгому сенсі, а лише її чисельним наближенням. Саме тому в умові роботи окремо ставиться вимога оцінити “розмитість” оцінки похибки. Під цим слід розуміти те, що різниця між сусідніми наближеннями може лише побічно відображати реальну помилку. У деяких випадках вона може бути дуже близькою до істинної похибки, а в деяких — помітно відрізнятися від неї. Отже, практична оцінка похибки є інформативною, але не абсолютно точною характеристикою відхилення від істинного значення.

Розмитість оцінки похибки проявляється в тому, що мала величина

$$|P_n(x) - P_{n-1}(x)|$$

зазвичай свідчить про стабілізацію результату, але сама по собі ще не гарантує ідеально малої абсолютної похибки. Це пов’язано з тим, що обидва сусідні многочлени можуть бути близькими між собою, але водночас разом відхилятися від точного значення функції. Проте на практиці, якщо обидві величини — і

$$|P_n(x) - P_{n-1}(x)|,$$

і

$$|f(x) - P_n(x)|$$

— є малими, то це свідчить про високу якість інтерполяції.

Графічний аналіз результатів є особливо наочним. Побудований графік дозволяє одночасно порівняти три залежності:

- точну функцію $f(x)$;
- інтерполяційний многочлен $P_n(x)$;
- абсолютну похибку $|f(x) - P_n(x)|$.

Якщо графіки функції та інтерполяції майже збігаються, це означає, що інтерполяція є якісною на всьому відрізку або на його більшій частині. Якщо ж графік похибки має локальні піки, то це вказує на ділянки, де точність інтерполяції погіршується. Зазвичай найбільші відхилення виникають поблизу кінців інтервалу або в тих місцях, де функція змінюється швидше. Отже, графік похибки є важливим не лише як ілюстрація, а і як інструмент аналізу стійкості та якості інтерполяції.

Особливий інтерес становить аналіз впливу степеня многочлена. Теоретично збільшення кількості вузлів повинно підвищувати точність у вибраній точці, оскільки використовується більше інформації про функцію. Проте на практиці це не завжди означає рівномірне покращення на всьому відрізку. При високих степенях інтерполяційного многочлена можуть виникати небажані осциляції, особливо на краях інтервалу. У цій роботі таблиця значень $P_n(x)$ дозволяє прослідкувати, чи дійсно збільшення степеня приводить до збіжності, чи навпаки виникають нестійкі коливання. Саме тому аналіз ряду значень для різних n є важливішим, ніж розгляд лише одного фінального результату $P_{0,n}(x)$.

Додатково програму було налагоджено на функції

$$f(x) = \sin x.$$

Цей етап має важливе методичне значення. Функція $\sin x$ є добре відомою, гладкою та зручною для перевірки чисельних методів. Якщо програма коректно інтерполуює $\sin x$, правильно формує таблиці значень, обчислює наближення різних степенів і відображає очікувану поведінку похибки, це підтверджує правильність реалізації рекурентної схеми Ейткена. Отже, тестування на $\sin x$ виконує роль верифікації програмної реалізації перед застосуванням до основної функції варіанту.

Важливо також зазначити, що сама природа функції істотно впливає на похибку інтерполяції. Для функції

$$f(x) = \sin(x^2)e^{-(x/2)^2}$$

поведінка є складнішою, ніж у випадку $\sin x$, оскільки одночасно присутні осциляції та експоненціальне затухання. Це означає, що похибка інтерполяції визначається не лише кількістю вузлів, а й локальними особливостями функції на відрізку. Саме тому побудова графіка та аналіз таблиці значень є необхідними: вони дозволяють побачити, наскільки добре інтерполяція адаптується до реальної форми функції.

Отже, результати роботи показали таке:

1. реалізована програма коректно виконує інтерполяцію за схемою Ейткена;
2. метод дає можливість обчислювати значення інтерполяційного многочлена без явного запису його алгебраїчного вигляду;
3. порівняння значень $P_n(x)$ для різних n дозволяє оцінити збіжність інтерполяційного процесу;
4. величина

$$|P_n(x) - P_{n-1}(x)|$$

є зручною практичною оцінкою похибки, але вона має розмитий характер і не повністю збігається з реальною похибкою;

5. графічне представлення результатів дає можливість виявити ділянки найбільшого відхилення та оцінити якість інтерполяції на всьому відрізку;
6. налагодження на функції $\sin x$ підтвердило правильність програмної реалізації методу.

Таким чином, проведений аналіз показує, що схема Ейткена є ефективним та зручним чисельним методом інтерполяції, а поєднання табличного, числового та графічного аналізу дозволяє не лише отримати результат, а й змістовно оцінити його точність та надійність.